

ALGORITMOS E COMPLEXIDADE

Semana 4

Atividade de Laboratório

Prática de Análise de Algoritmos

Objetivo da atividade

Observar, medir e comparar o comportamento de algoritmos conforme o tamanho da entrada aumenta. Ao final, você deverá ser capaz de relacionar resultados experimentais às classes de complexidade $O(1)$, $O(n)$, $O(n^2)$ e $O(\log n)$.

Aluno(a):
Gustavo Leal Lago

Dupla:
Caio Cesar

Data: 09 / 09 / 2026

Horário: 19h30 às 21h30

Regras de execução

Trabalhe em dupla, mas ambos devem compreender o código. Registre os resultados nas tabelas, responda às perguntas de análise e entregue o código-fonte junto com uma conclusão final. Não basta executar: o objetivo é explicar por que o algoritmo apresenta determinado comportamento.

Parte 1 - Contando operações

Você irá comparar três funções com padrões de crescimento diferentes. Compile e execute o programa abaixo alterando o valor de n .

```
#include <iostream>
using namespace std;

long long acessoDireto(int vetor[]) {
    long long operacoes = 0;
    int x = vetor[0];
    operacoes++;
    return operacoes;
}

long long somaVetor(int vetor[], int n) {
    long long operacoes = 0;
    long long soma = 0;

    for (int i = 0; i < n; i++) {
        soma += vetor[i];
        operacoes++;
    }
}
```

```

    return operacoes;
}

long long paresDeElementos(int n) {
    long long operacoes = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            operacoes++;
        }
    }

    return operacoes;
}

int main() {
    const int MAX = 10000;
    static int vetor[MAX];

    for (int i = 0; i < MAX; i++) {
        vetor[i] = i;
    }

    int n;
    cout << "Digite n: ";
    cin >> n;

    cout << "Acesso direto: " << acessoDireto(vetor) << endl;
    cout << "Soma do vetor: " << somaVetor(vetor, n) << endl;
    cout << "Pares de elementos: " << paresDeElementos(n) << endl;

    return 0;
}

```

A função somaVetor, pois realiza exatamente n operações.

1.1 Execução

Execute o programa quatro vezes, utilizando:

- n = 10
- n = 100
- n = 1.000
- n = 10.000

n	Acesso direto	Soma do vetor	Pares de elementos
10	1	10	100
100	1	100	10000
1.000	1	1000	1000000
10.000	1	10000	100000000

1.2 Análise

1. Qual função apresenta crescimento constante quando n aumenta?
2. Qual função cresce aproximadamente na mesma proporção de n? A função somaVetor, pois realiza exatamente n operações.
3. Qual função cresce muito mais rapidamente? O que ocorre quando n é multiplicado por 10?

Atividade de laboratório - Prática de Análise de Algoritmos

A função acessoDireto, pois realiza sempre apenas 1 operação, independentemente do tamanho de n.

A função paresDeElementos cresce muito mais rapidamente. Ela possui crescimento quadrático. Quando n é multiplicado por 10, a quantidade de operações é multiplicada por 100, pois:

$$(10n)^2 = 100n^2.$$

- Classifique as três funções utilizando O(1), O(n) e O(n²).
- Explique, com suas palavras, por que os três algoritmos apresentam comportamentos diferentes.

Os algoritmos apresentam comportamentos diferentes porque realizam quantidades diferentes de operações conforme o tamanho da entrada aumenta. O acesso direto executa uma única operação, por isso é O(1). A soma percorre o vetor uma vez, realizando n operações, sendo O(n). Já o algoritmo de pares possui dois laços de repetição aninhados, fazendo aproximadamente n² operações, sendo O(n²). Portanto, quanto maior for a entrada, maior será a diferença de desempenho entre eles.

Parte 2 - Busca linear x busca binária

Agora você irá comparar dois algoritmos que resolvem o mesmo problema: localizar um valor em um vetor. O vetor usado na busca binária deve estar ordenado.

```
int buscaLinear(int vetor[], int n, int valor, long long &comparacoes) {
    comparacoes = 0;

    for (int i = 0; i < n; i++) {
        comparacoes++;
        if (vetor[i] == valor) {
            return i;
        }
    }

    return -1;
}

int buscaBinaria(int vetor[], int n, int valor, long long &comparacoes) {
    int inicio = 0;
    int fim = n - 1;
    comparacoes = 0;

    while (inicio <= fim) {
        int meio = inicio + (fim - inicio) / 2;
        comparacoes++;

        if (vetor[meio] == valor) {
            return meio;
        }

        if (vetor[meio] < valor) {
            inicio = meio + 1;
        } else {
            fim = meio - 1;
        }
    }

    return -1;
}
```

2.1 Monte o programa de teste

Crie um vetor ordenado contendo valores de 0 até n - 1. No programa principal, execute as duas buscas para o mesmo valor e mostre:

- posição retornada pela busca linear;
- quantidade de comparações da busca linear;
- posição retornada pela busca binária;
- quantidade de comparações da busca binária.

2.2 Casos de teste obrigatórios

Para cada tamanho de vetor, faça pelo menos estes três testes:

Caso	Valor pesquisado	O que representa
A	0	Elemento na primeira posição
B	$n - 1$	Elemento na última posição
C	-1	Elemento inexistente

Utilize inicialmente $n = 100$ e $n = 1.000$. Se houver tempo, repita com $n = 10.000$.

n	Caso	Valor	Comp. linear	Comp. binária
100	A	0	1	6
100	B	$n - 1$	100	7
100	C	-1	100	6
1.000	A	0	1	9
1.000	B	$n - 1$	1000	10
1.000	C	-1	1000	9

2.3 Análise

6 - Quando o elemento pesquisado está na primeira posição do vetor. Nesse caso, é necessária apenas uma comparação.

7 - Quando o elemento está na última posição ou quando o elemento não existe no vetor. Nesses casos, praticamente todo o vetor precisa ser percorrido.

8 - Porque, no pior caso, ela precisa verificar cada um dos n elementos do vetor até encontrar o valor ou concluir que ele não existe.

9 - Porque a cada comparação a busca elimina aproximadamente metade dos elementos que ainda poderiam conter o valor. Assim, o número de comparações cresce de forma logarítmica.

- Em qual situação a busca linear realiza menos comparações?
- Em qual situação a busca linear se aproxima do pior caso?
- Por que a busca linear é classificada como $O(n)$ no pior caso?
- Por que a busca binária é classificada como $O(\log n)$?
- Qual condição o vetor precisa atender para que a busca binária possa ser utilizada?
- A busca binária é sempre a melhor escolha em qualquer situação? Justifique.

10 - O vetor precisa estar ordenado.

Parte 3 - Desafio integrador: Central de Consultas

Você irá reunir os conceitos anteriores em um pequeno programa completo. Imagine um sistema que mantém códigos numéricos de produtos em ordem crescente e precisa localizar rapidamente um código informado pelo usuário.

Desafio

Crie um programa que permita comparar, na mesma execução, o resultado da busca linear e da busca binária sobre o mesmo conjunto de códigos.

3.1 Requisitos obrigatórios

- Crie um vetor ordenado de códigos inteiros.
- Utilize inicialmente $n = 1.000$.
- Solicite ao usuário um código para pesquisa.
- Pesquise o código utilizando busca linear.
- Pesquise o mesmo código utilizando busca binária.
- Mostre se o código foi encontrado e, quando encontrado, sua posição.
- Mostre a quantidade de comparações realizadas por cada algoritmo.

11 - Não. A busca binária é muito eficiente para vetores ordenados, mas exige que os dados estejam ordenados. Para uma busca simples em um vetor pequeno ou não ordenado, a busca linear pode ser mais adequada e simples de implementar. Além disso, a busca binária depende da possibilidade de acessar diretamente posições do vetor.

19. Repita o teste para um valor existente e para um valor inexistente.

3.2 Saída esperada

Sua saída pode seguir este formato (os valores numéricos abaixo são apenas ilustrativos):

```
Digite o código: 782
```

```
Busca Linear
Encontrado: sim
Posição: 782
Comparações: ...
```

```
Busca Binária
Encontrado: sim
Posição: 782
Comparações: ...
```

3.3 Extensão para quem terminar antes

- Repita o experimento com $n = 100$, 10.000 e 100.000 .
- Organize os resultados em uma tabela.
- Explique como a diferença entre as duas buscas evolui conforme n aumenta.

Entrega

Ao final da atividade, entregue os seguintes itens na aba de trabalhos da disciplina no SAVA:

- códigos utilizado(s) nos experimentos;
- tabelas de resultados preenchidas;
- respostas das perguntas de análise;
- programa da Central de Consultas;
- conclusão final em um pequeno parágrafo.

A atividade demonstra que o tamanho da entrada influencia diretamente o desempenho dos algoritmos. Enquanto a função de pares chegou a 100.000.000 operações. Na busca, com $n = 1.000$, a busca linear precisou realizar até 1.000 comparações, enquanto a busca binária precisou de aproximadamente 10. Isso mostra que algoritmos $O(\log n)$ podem ser muito mais eficientes que algoritmos $O(n)$ para grandes quantidades de dados. Portanto, conhecer a complexidade ajuda a escolher o algoritmo mais adequado para cada situação.

(Você pode colocar tudo no mesmo arquivo docx, txt ou pdf e enviar. Ou colocar em um repositório git público e enviar o link)

Conclusão final - responda em 5 a 8 linhas

O que os experimentos realizados demonstram sobre a relação entre o tamanho da entrada e a escolha do algoritmo? Use pelo menos dois resultados obtidos durante a atividade para justificar sua resposta.

Checklist antes de entregar

☒	Parte 1 executada com os quatro valores de n .
☒	Tabela da Parte 1 preenchida.
☒	Busca linear e binária testadas em pelo menos dois tamanhos.
☒	Três casos de busca testados: início, fim e inexistente.
☒	Central de Consultas funcionando.
☒	Conclusão final escrita.

